Lab 05: A "gentle" Introduction to Support Vector Machines in R

Andrés Oswaldo Calderón Romero, PhD.

October 29, 2026

1 Introduction

Support Vector Machines (SVMs) offer a powerful approach to binary classification by attempting to find a **hyperplane** in a feature space that "best" separates two classes. While perfect linear separation is often impossible in the original feature space, SVMs extend this concept by softening the separation requirements and using the **kernel trick** to enlarge the feature space, making separation more likely.

2 The Kernel Trick

When classes are not linearly separable, SVMs use **kernel functions** to project data into a higher-dimensional space where a linear hyperplane can separate them. Common kernels include:

- **Polynomial**: $K(x, x') = (1 + \langle x, x' \rangle)^d$
- **Radial Basis Function (RBF)**: $K(x, x') = \exp(-\gamma ||x - x'||^2)$
- **Hyperbolic Tangent**: $K(x, x') = \tanh(\kappa_1 ||x - x'|| + \kappa_2)$

3 Implementation in R

Although there are a number of great packages that implement SVMs (e.g., e1071 (Meyer et al. 2019) and svmpath (Hastie 2016)), we'll focus on the most flexible implementation of SVMs in R: kernlab (Karatzoglou et al. 2004). We'll also use caret for tuning SVMs and pre-processing. In this chapter, we'll explicitly load the following packages:

```r
# Helper packages
library(dplyr)    # for data wrangling
library(ggplot2)  # for awesome graphics
library(modeldata)  # for data access and splitting

# Modeling packages
library(caret)    # for classification and regression training
library(kernlab)  # for fitting SVMs

# Model interpretability packages
library(pdp)      # for partial dependence plots, etc.
library(vip)      # for variable importance plots
```

Listing 1: Package Initialization: Loading essential libraries for data wrangling, visualization, and SVM modelling.

To illustrate the basic concepts of fitting SVMs we'll use a mix of simulated data sets as well as the employee attrition data. The code for generating the simulated data sets and figures in this chapter are available on the book website. In the employee attrition example our intent is to predict on Attrition (coded as "Yes"/"No"). As in previous labs, we'll set aside 30% of the data for assessing generalizability.

```r
# Load attrition data
df <- attrition %>%
  mutate_if(is.ordered, factor, ordered = FALSE)

# Create training (70%) and test (30%) sets
set.seed(42)  # for reproducibility
churn_split <- initial_split(df, prop = 0.7, strata = "Attrition")
churn_train <- training(churn_split)
churn_test  <- testing(churn_split)
```

Listing 2: Data Preparation: Formatting the "attrition" dataset and partitioning it into 70% training and 30% testing sets.

Notice how each of these kernel functions include hyperparameters that need to be tuned. For example, the polynomial kernel includes a degree term d and a scale parameter γ. Similarly, the radial basis kernel includes a γ parameter related to the inverse of the σ parameter of a normal distribution. In R, you can use caret's `getModelInfo()` to extract the hyperparameters from various SVM implementations with different kernel functions, for example:

```r
# Linear (i.e., soft margin classifier)
caret::getModelInfo("svmLinear")$svmLinear$parameters
##   parameter   class label
## 1         C numeric  Cost

# Polynomial kernel
caret::getModelInfo("svmPoly")$svmPoly$parameters
##   parameter   class             label
## 1    degree numeric Polynomial Degree
## 2     scale numeric             Scale
## 3         C numeric              Cost

# Radial basis kernel
caret::getModelInfo("svmRadial")$svmRadial$parameters
##   parameter   class label
## 1     sigma numeric Sigma
## 2         C numeric  Cost
```

Listing 3: Hyperparameter Discovery: Identifying tunable parameters (e.g., C, *degree*, and σ) for Linear, Polynomial, and RBF kernels.

We utilize the `kernlab` package for fitting SVMs and `caret` for hyperparameter tuning. The plotting results can be seen in Figure 1.

```r
# Example: Training an SVM with a Radial Basis Kernel
# 'attrition' dataset assumed to be pre-loaded

# Tune an SVM with radial basis kernel

set.seed(42)
churn_svm <- train(
    Attrition ~ .,
    data = churn_train,
    method = "svmRadial",
    preProcess = c("center", "scale"),
    trControl = trainControl(method = "cv", number = 10),
    tuneLength = 10
)

# View tuning results
print(churn_svm$results)

#          sigma      C  Accuracy     Kappa  AccuracySD   KappaSD
# 1  0.009788409   0.25 0.8395141 0.0000000 0.004205199 0.0000000
# 2  0.009788409   0.50 0.8395141 0.0000000 0.004205199 0.0000000
# 3  0.009788409   1.00 0.8590082 0.1906199 0.015478150 0.1114105
# 4  0.009788409   2.00 0.8833002 0.4301047 0.027982904 0.1551574
# 5  0.009788409   4.00 0.8842998 0.4693976 0.029904716 0.1644876
# 6  0.009788409   8.00 0.8794357 0.4750025 0.030505379 0.1589090
# 7  0.009788409  16.00 0.8706501 0.4515783 0.031627149 0.1538052
# 8  0.009788409  32.00 0.8696886 0.4537813 0.027796066 0.1413571
# 9  0.009788409  64.00 0.8687177 0.4518817 0.029981421 0.1452463
# 10 0.009788409 128.00 0.8687177 0.4518817 0.029981421 0.1452463

# Plot results
g <- ggplot(churn_svm) + theme_light()
plot(g)
```

Listing 4: SVM Implementation: Training a Radial Basis Function (RBF) model with 10-fold cross-validation and hyperparameter optimization.

4 Autonomous Work

Reproduce the Support Vector Machine (SVM) implementation using the "Advertisement" dataset as detailed in Section 3 of Lab 05. Specifically, you are required to:

1. **Data Partitioning:** Load the dataset and perform a 75/25 training/testing split.
2. **Feature Scaling:** Standardize the "Age" and "EstimatedSalary" variables.
3. **Model Fitting:** Train an SVM classifier using the `ksvm` function from the `kernlab` package or the `train` function from `caret`.
4. **Kernel Evaluation:** Evaluate model performance across various kernels (Linear, Polynomial, and RBF) and report the resulting accuracy for each.

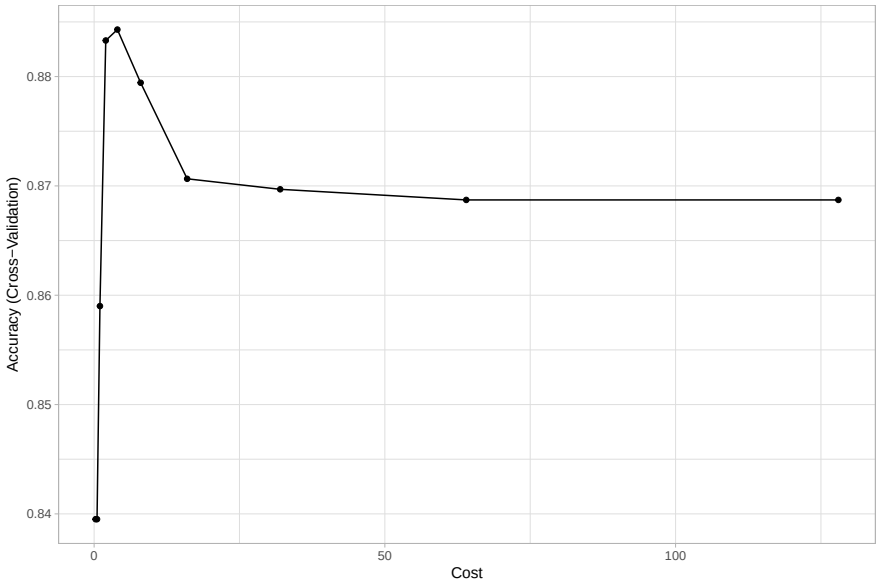


Figure 1: Plotting the results, we see that smaller values of the cost parameter ($C \approx 2$–8) provide better cross-validated accuracy scores for these training data.

5. **Performance Analysis:** Generate a confusion matrix and calculate validation metrics. Compare these results with the performance of the Decision Tree model developed in the previous lab.

Note: You can use the programming language of your choice.

References

Hastie, Trevor (2016). *svmpath: The SVM Path Algorithm*. R package version 0.96. URL: `https://CRAN.R-project.org/package=svmpath`.

Karatzoglou, Alexandros et al. (2004). "kernlab - An S4 Package for Kernel Methods in R". In: *Journal of Statistical Software* 11.9, pp. 1–20. DOI: `10.18637/jss.v011.i09`.

Meyer, David et al. (2019). *e1071: Misc Functions of the Department of Statistics, Probability Theory Group (Formerly: E1071), TU Wien*. R package version 1.7-2. URL: `https://CRAN.R-project.org/package=e1071`.